EventHive

Serverless Campus Event Registration System on AWS

Digital Assignment 3 – AWS Cloud Project Work

Submitted by
Rishab C V
Reg. No. 24BCE5296
B.Tech Computer Science and Engineering

Vellore Institute of Technology, Chennai
October 2026

1. Abstract

EventHive is a cloud-native web application that lets students browse campus events, reserve a seat and receive a ticket instantly, while organisers manage events and track registrations through a live analytics dashboard. The system is built entirely from managed, serverless AWS services – Amazon S3, CloudFront, API Gateway, AWS Lambda, DynamoDB, SNS and CloudWatch – and is provisioned as Infrastructure as Code with AWS SAM. A key design goal was correctness under concurrency: seat booking uses DynamoDB transactions so an event can never be overbooked, even when many students register at the same moment. Because no component runs continuously, the application costs effectively nothing when idle.

2. Problem Statement

Campus clubs usually collect registrations through Google Forms or spreadsheets. These have no seat limits, allow duplicate entries, give no instant confirmation, and offer organisers no real-time view of how events are filling. A traditional fix – a web server on a virtual machine – costs money every hour it runs, even when nobody is using it, and must be patched and scaled manually.

Objectives

- Allow students to view events and book a seat with instant ticket generation.
- Strictly enforce seat capacity and prevent duplicate registrations.
- Give organisers an admin dashboard with fill-rate and category analytics.
- Use only serverless AWS services so the system scales automatically and has near-zero idle cost.
- Define the entire infrastructure as code so it can be deployed or torn down with one command.

3. System Architecture

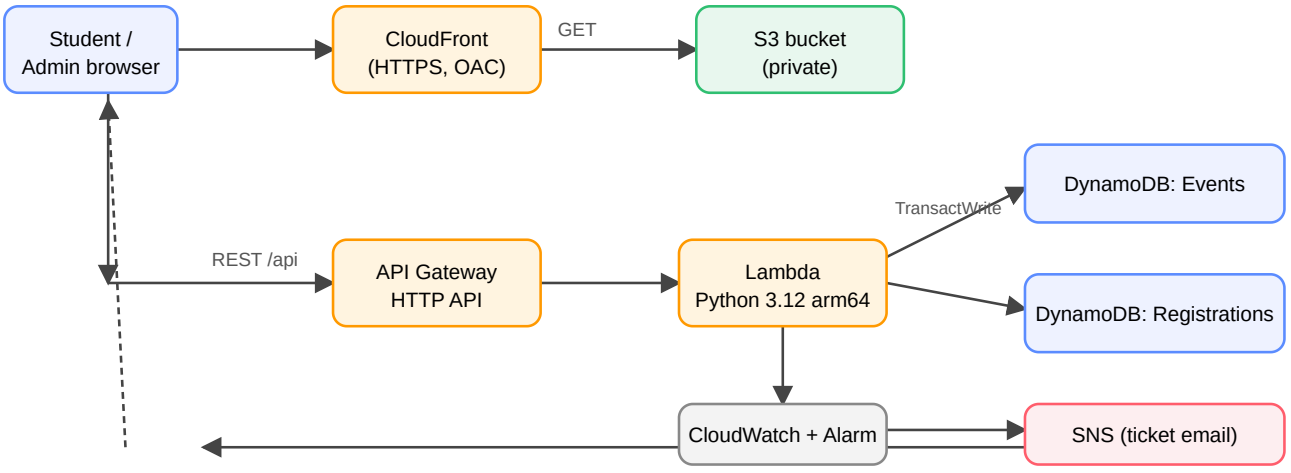


Figure 1 – EventHive architecture

Layer	AWS Service	Role / reason for choice
Frontend hosting	Amazon S3 + CloudFront	Static site stored in a private bucket; CloudFront serves it over HTTPS with global caching. Origin Access Control means the bucket is never public.
API	API Gateway (HTTP API)	Lower cost and latency than REST APIs; built-in CORS and throttling (10 req/s, burst 20).
Compute	AWS Lambda (Python 3.12, arm64)	Runs only on request. Graviton (arm64) is about 20% cheaper than x86.
Database	Amazon DynamoDB (on-demand)	Two tables – Events and Registrations. Millisecond latency, pay-per-request, no cost when idle.
Notifications	Amazon SNS	Sends ticket confirmation emails and forwards CloudWatch alarm alerts.
Monitoring	Amazon CloudWatch	Lambda logs (7-day retention) and an alarm on Lambda errors.
Security	IAM	Least-privilege role: CRUD on only the two tables and publish on only one topic.
Provisioning	AWS SAM / CloudFormation	The whole stack is declared in template.yaml.

4. Database Design

Table	Partition key	Sort key	Attributes
Events	eventId (S)	–	title, venue, date, category, capacity, registered, createdAt
Registrations	eventId (S)	email (S)	name, regno, ticket, registeredAt

Using *(eventId, email)* as the composite key of Registrations makes duplicate bookings impossible by design and lets the attendee list for an event be fetched with a single Query.

5. Implementation

5.1 API endpoints

Method	Path	Access	Purpose
GET	/events	Public	List all events sorted by date
POST	/events	Admin	Create event
DELETE	/events/{id}	Admin	Delete event and its registrations
POST	/events/{id}/register	Public	Book a seat, returns ticket ID

GET	/events/{id}/registrations	Admin	Attendee list
GET	/stats	Admin	Totals, fill rate per event and per category

5.2 Atomic seat booking

The core of the system is a single DynamoDB **TransactWriteItems** call containing two operations. Either both succeed or both are rolled back:

```
ddb.meta.client.transact_write_items(TransactItems=[
  {"Update": {"TableName": events.name, "Key": {"eventId": event_id},
     "UpdateExpression": "SET registered = registered + :one",
     "ConditionExpression": "attribute_exists(eventId) AND registered < #cap",
     "ExpressionAttributeNames": {"#cap": "capacity"},
     "ExpressionAttributeValues": {":one": 1}}},
  {"Put": {"TableName": regs.name, "Item": {...},
     "ConditionExpression": "attribute_not_exists(email)"}}
])
```

If the event is full the first condition fails; if the student already registered the second fails. The Lambda reads the cancellation reasons and returns a clear 409 message (“event is full” or “already registered”). Without this, two students clicking “Register” on the last seat at the same instant could both get in.

5.3 Infrastructure as Code

template.yaml declares the two DynamoDB tables, SNS topic and optional email subscription, HTTP API with CORS and throttling, the Lambda function with least-privilege policies, a CloudWatch log group and error alarm, the private S3 bucket, a CloudFront distribution with Origin Access Control and the bucket policy. deploy.sh runs *sam build* and *sam deploy*, uploads the frontend with the generated API URL, and *deploy.sh destroy* removes everything. The template passes *cfn-lint* with no errors.

5.4 Local development without AWS cost

local_server.py starts **moto**, which emulates DynamoDB and SNS in memory, and then serves the frontend and routes /api requests into the exact same Lambda handler used in production. This made it possible to build and test the whole application without spending any AWS credits.

6. Results

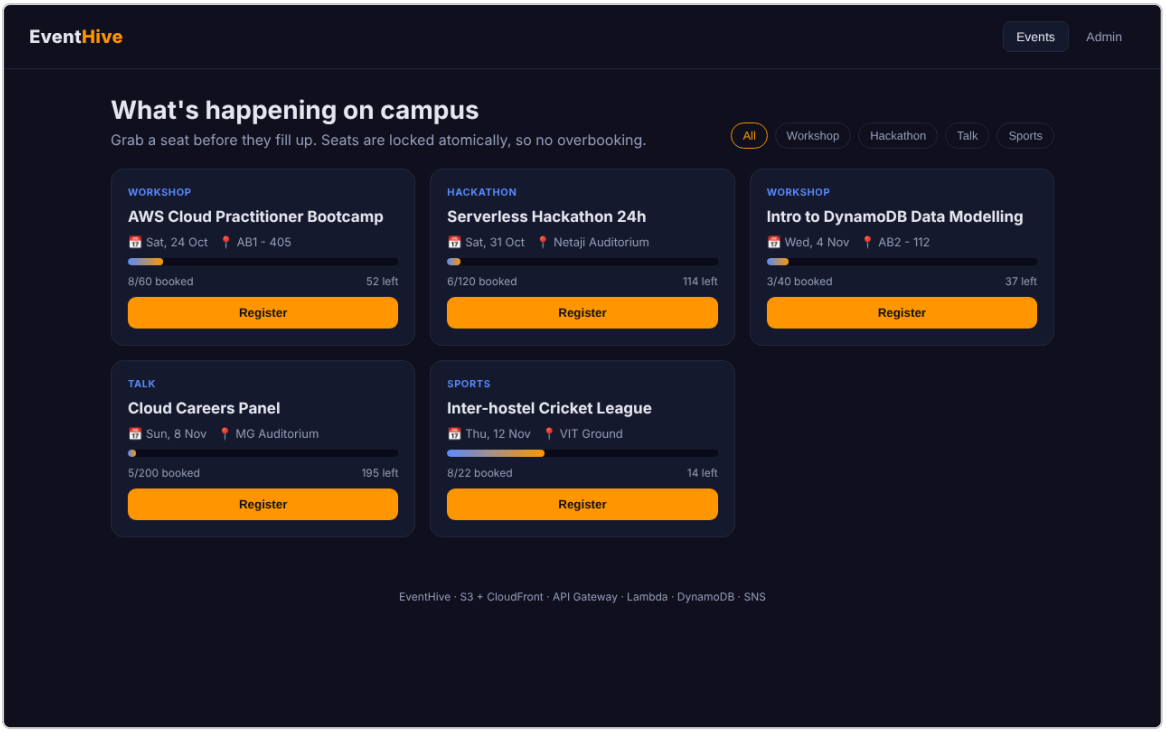


Figure 2 – Student view: event cards with live seat availability and category filters

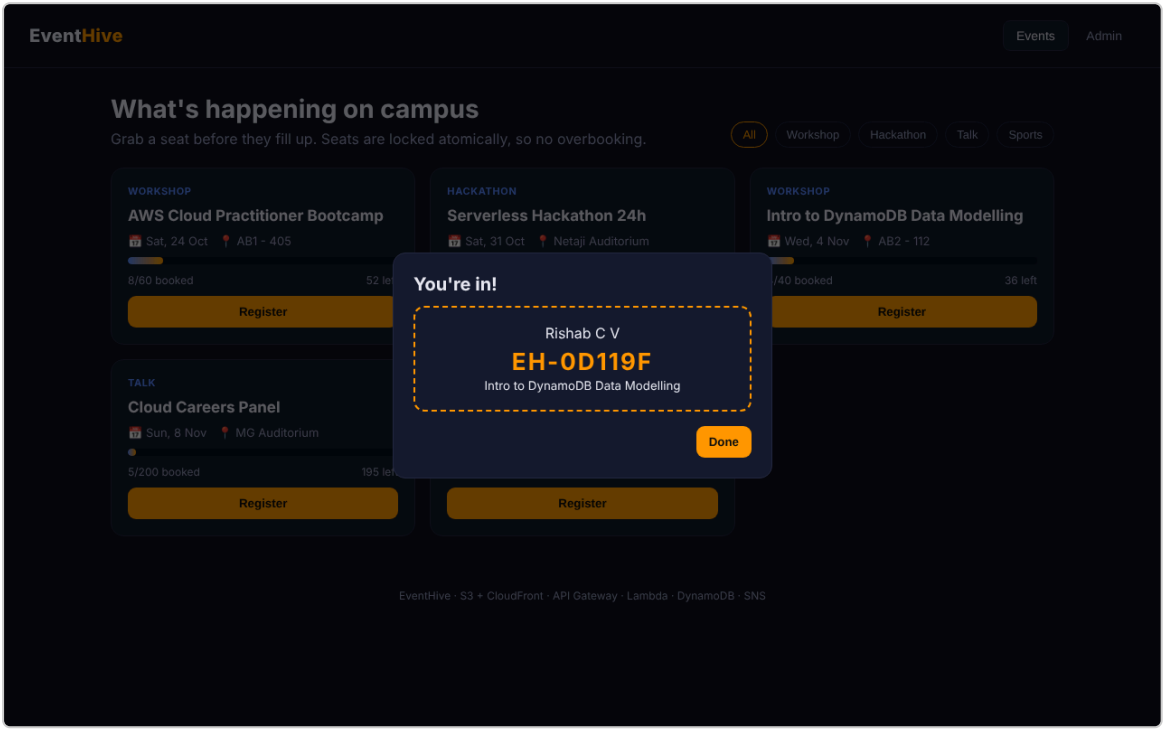


Figure 3 – Instant ticket after a successful registration

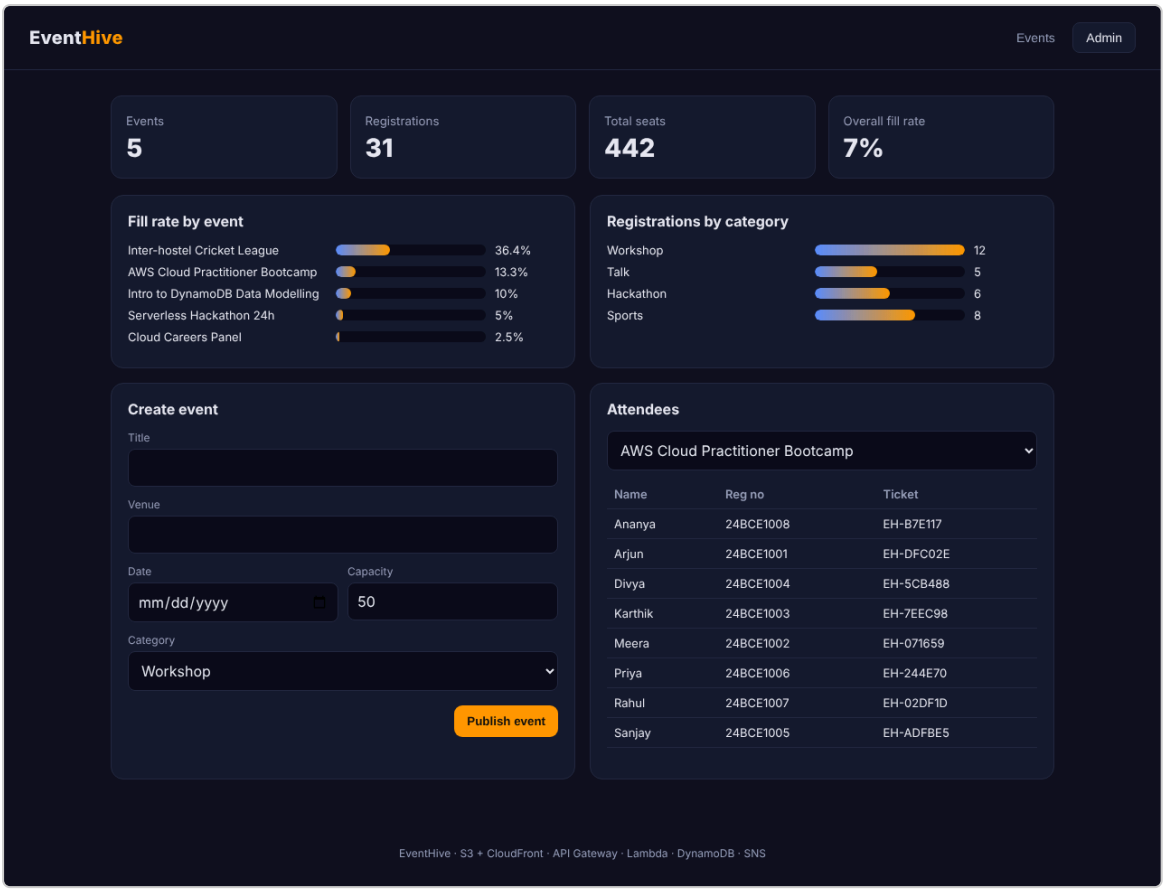


Figure 4 – Admin dashboard: KPIs, fill rate by event, category breakdown, event creation and attendee list

7. Testing

Eight automated tests run against the Lambda handler with moto-backed DynamoDB. The most important one, *test_no_overbooking_under_load*, fires 20 concurrent registrations at an event with 2 seats and checks that exactly 2 succeed and the counter ends at 2.

Test	Checks
test_create_needs_admin	Event creation without admin key returns 403
test_register_and_ticket	Successful booking returns an EH- ticket
test_duplicate_blocked	Same email twice returns 409
test_capacity_enforced	Third booking on a 2-seat event returns 409 “full”
test_no_overbooking_under_load	20 parallel requests, exactly 2 succeed
test_bad_email	Invalid email returns 400
test_unknown_event	Booking a non-existent event returns 404
test_stats	Fill-rate calculation is correct

```
test_bad_email (__main__.EventHiveTest.test_bad_email) ... ok
test_capacity_enforced (__main__.EventHiveTest.test_capacity_enforced) ... ok
test_create_needs_admin (__main__.EventHiveTest.test_create_needs_admin) ... ok
test_duplicate_blocked (__main__.EventHiveTest.test_duplicate_blocked) ... ok
test_no_overbooking_under_load (__main__.EventHiveTest.test_no_overbooking_under_load) ... ok
test_register_and_ticket (__main__.EventHiveTest.test_register_and_ticket) ... ok
test_stats (__main__.EventHiveTest.test_stats) ... ok
test_unknown_event (__main__.EventHiveTest.test_unknown_event) ... ok

----------------------------------------------------------------------
Ran 8 tests in 1.394s

OK
```

8. Cost Analysis

Service	Expected usage (class demo)	Monthly cost
Lambda	< 10,000 invocations	Free tier (1M requests)
API Gateway HTTP API	< 10,000 calls	≈ $0.01
DynamoDB on-demand	< 1 MB, < 10,000 ops	Free tier (25 GB)
S3	2 files, < 50 KB	≈ $0.00
CloudFront	< 1 GB	Free tier (1 TB)

SNS email	< 100 emails	Free tier (1,000)
Total		**≈ $0 – $0.05**

For comparison, a single t3.micro EC2 instance with a public IPv4 address running 24/7 costs roughly $11–12 per month in the same account, whether or not anyone uses it. The serverless design removes this idle cost completely.

9. Security Measures

- S3 bucket has all public access blocked; only CloudFront can read it via Origin Access Control (SigV4).
- HTTPS enforced through CloudFront (redirect-to-https).
- Lambda IAM role follows least privilege using SAM policy templates.
- Admin key passed as a NoEcho CloudFormation parameter, never hard-coded in the deployed function.
- API Gateway throttling limits abuse and protects against unexpected bills.
- Server-side validation of email format, required fields and capacity range.

10. Conclusion and Future Work

EventHive shows how a real campus problem can be solved with a fully serverless AWS architecture that is secure, scales automatically, guarantees correctness under concurrent load, and costs almost nothing to run. Defining the system in SAM makes it reproducible and easy to remove, and the moto-based local environment allowed the full application to be developed and tested without consuming AWS credits.

Future enhancements: Amazon Cognito for proper user and organiser login; QR-code tickets with venue check-in; DynamoDB Streams to auto-promote students from a waitlist; AWS WAF in front of CloudFront; and a CI/CD pipeline with GitHub Actions.

11. References

1. AWS Lambda Developer Guide – docs.aws.amazon.com/lambda
2. Amazon DynamoDB Developer Guide: Transactions – docs.aws.amazon.com/amazondynamodb
3. Amazon API Gateway HTTP APIs – docs.aws.amazon.com/apigateway
4. Restricting access to an S3 origin with Origin Access Control – docs.aws.amazon.com/cloudfront
5. AWS Serverless Application Model (SAM) – docs.aws.amazon.com/serverless-application-model
6. Moto: Mock AWS Services – github.com/getmoto/moto